Яковець Арина, ФФ-41

ЛАБОРАТОРНА РОБОТА №3. Чисельне розв'язання нелінійних рівнянь та систем

Варіант 15

Мета: Закріплення знань із застосування методів чисельного розв'язання нелінійних алгебраїчних рівнянь та систем.

15. а) Визначити корінь рівняння $e^x - x - 2 = 0$ на інтервалі $[-2.4;-1.6]$ методом Ньютона з точністю 10^{-4}. Перевірити отриманий корінь та вивести кількість ітерацій;

Код реалізації:

```
import math

a = -2.4
b = -1.6
eps = 1e-4

print("Дано:")
print("F(x) = e^x - x - 2")
print("інтервал =", [a,b])
print("ε =", eps)

Fa = math.exp(a) - a - 2
Fb = math.exp(b) - b - 2

print("\nПеревірка існування кореня:")
print("F(a) =", Fa)
print("F(b) =", Fb)

print("F(a)*F(b) =", Fa*Fb)
if Fa*Fb < 0:
    print("F(a)*F(b) < 0, на відрізку є корінь")
else:
    print("Кореня на відрізку може не бути")

#другі похідні
F2a = math.exp(a)
F2b = math.exp(b)

print("\nПеревірка умови вибору початкового наближення:")
print("F''(x) = e^x")
print("F(a)*F''(a) =", Fa*F2a)
print("F(b)*F''(b) =", Fb*F2b)
if Fa*F2a > 0:
    x = a
    print("F(a)*F''(a) > 0, x0 =", x)
else:
    x = b
    print("F(b)*F''(b) > 0, x0 =", x)

k = 0
print("\nІтерації методу Ньютона")
print("k   x(k)        F(x)")

while True:
    F = math.exp(x) - x - 2
    dF = math.exp(x) - 1

    print(k,"  ", round(x,4),"    ", round(F,4))

    if abs(F) < eps:
        break

    x = x - F/dF
    k += 1

print("\nКорінь =", round(x,4))
print("Перевірка F(x) =", math.exp(x) - x - 2)
print("Кількість ітерацій =", k)
```

Результат:

```
Дано:
F(x) = e^x - x - 2
інтервал = [-2.4, -1.6]
ε = 0.0001

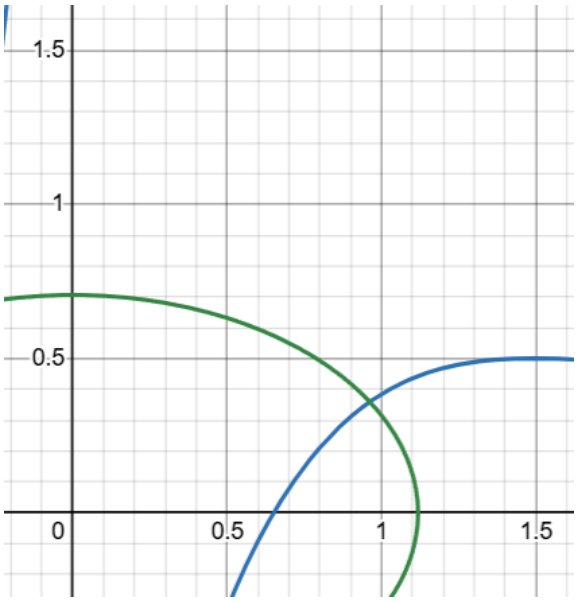
Перевірка існування кореня:
F(a) = 0.49071795328941237
F(b) = -0.19810348200534444
F(a)*F(b) = -0.09721293522916856
F(a)*F(b) < 0, на відрізку є корінь

Перевірка умови вибору початкового наближення:
F''(x) = e^x
F(a)*F''(a) = 0.04451692836478502
F(b)*F''(b) = -0.039996403219495914
F(a)*F''(a) > 0, x0 = -2.4

Ітерації методу Ньютона
k   x(k)         F(x)
0    -2.4      0.4907
1    -1.8603     0.0159
2    -1.8414     0.0

Корінь = -1.8414
Перевірка F(x) = 2.7925940435657992e-05
Кількість ітерацій = 2
```

б) Розв'язати систему рівнянь $\begin{cases} tg(xy+0.4)=x^2 \\ 0.8x^2+2y^2=1 \end{cases} (x>0, y>0)$ з точністю 10^{-5} методом Ньютона. Початкове наближення визначати графічно. Вивести кількість ітерацій;



Код реалізації:

```
import sympy as sp
eps = 1e-5

x, y = sp.symbols('x y')

F1 = sp.tan(x*y + 0.4) - x**2
F2 = 0.8*x**2 + 2*y**2 - 1

F1x = sp.diff(F1, x)
F1y = sp.diff(F1, y)
F2x = sp.diff(F2, x)
F2y = sp.diff(F2, y)

#початкове наближення з графіку
xn = 1.0
yn = 0.3

k = 0
print("\nk   x          y          F1         F2")

while True:
    f1 = float(F1.subs({x:xn, y:yn}))
    f2 = float(F2.subs({x:xn, y:yn}))

    print(k," ", round(xn,5)," ", round(yn,5)," ", round(f1,5),"  ", round(f2,5))

    a11 = float(F1x.subs({x:xn, y:yn}))
    a12 = float(F1y.subs({x:xn, y:yn}))
    a21 = float(F2x.subs({x:xn, y:yn}))
    a22 = float(F2y.subs({x:xn, y:yn}))

    det = a11*a22 - a12*a21

    dx = (-f1*a22 + a12*f2)/det
    dy = (f1*a21 - a11*f2)/det

    xn += dx
    yn += dy
    k += 1

    if max(abs(dx), abs(dy)) < eps:
        break
```

```
print("\nРозв'язок:")
print("x =", round(xn,5))
print("y =", round(yn,5))
print("\nКількість ітерацій =", k)
```

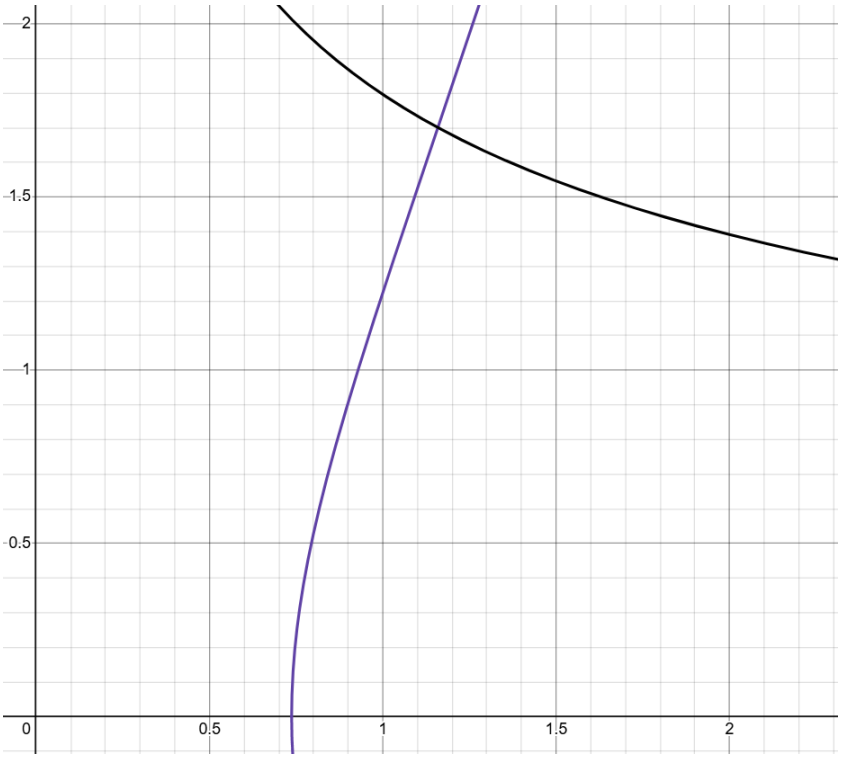
Результат:

```
k   x          y          F1         F2
0   1.0   0.3   -0.15771    -0.02
1   0.96569   0.36241   -0.00101    0.00873
2   0.96196   0.36037   2e-05    2e-05
3   0.96195   0.36036   0.0    0.0

Розв'язок:
x = 0.96195
y = 0.36036

Кількість ітерацій = 4
```

в) Розв’язати систему рівнянь $\begin{cases} 2.5x^3 - y^2 - 1 = 0 \\ xy^3 - y - 4 = 0 \end{cases}$ з точністю 10^{-5} методом простої ітерації;



Код реалізації:

```python
import sympy as sp
eps = 1e-5
x, y = sp.symbols('x y')

F1 = 2.5*x**3 - y**2 - 1
F2 = x*y**3 - y - 4

F1x = sp.diff(F1, x)
F1y = sp.diff(F1, y)
F2x = sp.diff(F2, x)
F2y = sp.diff(F2, y)

#початкове наближення з графіку
xn = 1.2
yn = 1.7

a11 = float(F1x.subs({x:xn, y:yn}))
a12 = float(F1y.subs({x:xn, y:yn}))
a21 = float(F2x.subs({x:xn, y:yn}))
a22 = float(F2y.subs({x:xn, y:yn}))

#коеф
alpha, beta, gamma, delta = sp.symbols('alpha beta gamma delta')

#система 3.25
sol = sp.solve([
    1 + alpha*a11 + beta*a21,
    alpha*a12 + beta*a22,
    gamma*a11 + delta*a21,
    1 + gamma*a12 + delta*a22
], [alpha, beta, gamma, delta])

alpha = float(sol[alpha])
beta  = float(sol[beta])
gamma = float(sol[gamma])
delta = float(sol[delta])

print("Початкове наближення:")
print("x0 =", xn)
print("y0 =", yn)

print("\nПохідні в точці x0, y0:")
print("F1x =", round(a11, 5))
print("F1y =", round(a12, 5))
```

```
print("F2x =", round(a21, 5))
print("F2y =", round(a22, 5))

print("\nКоефіцієнти:")
print("alpha =", round(alpha, 5))
print("beta  =", round(beta, 5))
print("gamma =", round(gamma, 5))
print("delta =", round(delta, 5))

k = 0
print("\nk   x          y          F1          F2")

while True:

    f1 = float(F1.subs({x:xn, y:yn}))
    f2 = float(F2.subs({x:xn, y:yn}))

    print(k," ",round(xn, 5)," ",round(yn, 5)," ",round(f1, 5)," ",round(f2, 5))

    x_new = xn + alpha*f1 + beta*f2
    y_new = yn + gamma*f1 + delta*f2

    dx = x_new - xn
    dy = y_new - yn

    xn = x_new
    yn = y_new
    k += 1

    if max(abs(dx), abs(dy)) < eps:
        break
#перевірка
f1 = float(F1.subs({x:xn, y:yn}))
f2 = float(F2.subs({x:xn, y:yn}))

print("\nРозв'язок:")
print("x =", round(xn, 5))
print("y =", round(yn, 5))

print("\nПеревірка:")
print("F1 =", f1)
print("F2 =", f2)

print("\nКількість ітерацій =", k)
```

Результат:

```
Початкове наближення:
x0 = 1.2
y0 = 1.7

Похідні в точці x0, y0:
F1x = 10.8
F1y = -3.4
F2x = 4.913
F2y = 9.404

Коефіцієнти:
alpha = -0.07951
beta  = -0.02875
gamma = 0.04154
delta = -0.09132

k    x          y          F1          F2
0    1.2    1.7    0.43    0.1956
1    1.16019    1.7    0.01411    -0.0
2    1.15906    1.70059    0.0008    -0.00021
3    1.15901    1.70064    4e-05    -2e-05

Розв'язок:
x = 1.159
y = 1.70064

Перевірка:
F1 = 2.0232410147258406e-06
F2 = -1.2054074831624462e-06

Кількість ітерацій = 4
```

г*) Розв’язати систему рівнянь $\begin{cases} x^2 + y^2 + z^2 - 14 = 0 \\ 2x^2 + y^2 - 4z + 4 = 0 \\ 3x^2 - 4y + z^2 - 4 = 0 \end{cases}$ з початковим наближенням $(1;2;3)$ та точністю 10^{-3}.

Код реалізації:

```
print(
import sympy as sp
eps = 1e-3

x, y, z = sp.symbols('x y z')

f1 = x**2 + y**2 + z**2 - 14
f2 = 2*x**2 + y**2 - 4*z + 4
f3 = 3*x**2 - 4*y + z**2 - 4

# вектор-функція
F = sp.Matrix([f1, f2, f3])

# матриця Якобі
W = F.jacobian([x, y, z])

xn = 1.0
yn = 2.0
zn = 3.0

k = 0

print("Початкове наближення:")
print("x0 =", xn)
print("y0 =", yn)
print("z0 =", zn)

while True:
    #F(xn, yn, zn)
    F_val = F.subs({x: xn, y: yn, z: zn})
    F_num = sp.Matrix([float(F_val[0]), float(F_val[1]), float(F_val[2])])

    #матриця Якобі в точці
    W_val = W.subs({x: xn, y: yn, z: zn})
    W_num = sp.Matrix([
        [float(W_val[0,0]), float(W_val[0,1]), float(W_val[0,2])],
        [float(W_val[1,0]), float(W_val[1,1]), float(W_val[1,2])],
        [float(W_val[2,0]), float(W_val[2,1]), float(W_val[2,2])]
    ])

    #визначник матриці Якобі
    detW = float(W_num.det())

    #обернена матриця
    W_inv = W_num.inv()
```

```
    #формула Ньютона: X_(k+1) = X_k - W^(-1)(X_k) * F(X_k)
    delta = W_inv * F_num

    x_new = xn - float(delta[0])
    y_new = yn - float(delta[1])
    z_new = zn - float(delta[2])

    dx = abs(x_new - xn)
    dy = abs(y_new - yn)
    dz = abs(z_new - zn)


    xn = x_new
    yn = y_new
    zn = z_new
    k += 1

    if max(dx, dy, dz) < eps:
        break

#перевірка
F_check = F.subs({x: xn, y: yn, z: zn})
F_check_num = sp.Matrix([float(F_check[0]), float(F_check[1]), float(F_check[2])])

print("Розв'язок:")
print("x =", round(xn, 6))
print("y =", round(yn, 6))
print("z =", round(zn, 6))

print("\nПеревірка:")
print(F_check_num)

print("\nКількість ітерацій =", k)
```

Результат:

```
Початкове наближення:
x0 = 1.0
y0 = 2.0
z0 = 3.0
Розв'язок:
x = 1.206369
y = 2.112256
z = 2.84307

Перевірка:
Matrix([[5.91481690292994e-8], [1.18288861372662e-7], [1.77429066994250e-7]])

Кількість ітерацій = 3
```